BUILD A CUBESAT

VERSION 0.4 · 2026-10-29

Build a CubeSat – Project Overview

Consolidated project reference

CONTENTS

0	Authority and Scope	2
1	In One Paragraph	2
2	Why BAC Exists	2
3	Project Principles	3
4	Historical Context	4
5	System Architecture	4
6	Maturity and Qualification	5
7	Products and Commercial Model	5
8	Open-Source Model	6
9	Community, Documentation and Education	6
10	BAC and nihil.space	6
11	Development Direction	7
12	What This Overview Deliberately Omits	7
13	Canonical References	8
14	Summary	8
15	Version History	9

Space for everyone. This document is the durable orientation reference for Build a CubeSat (BAC): what the project is, why it exists, how its major parts fit together, how it relates to nihil.space, and where authoritative detail lives.

0 Authority and Scope

This overview synthesizes the project but is not the authority for specialist rules. When documents disagree, use the source closest to the subject:

1. **BAC Identity Guide** – brand, voice and tone, typography, color, visual language, imagery, and the Ops/Mission communication modes.
2. **BAC Interface Design Guide** – interface behavior, accessibility, interaction, semantic states, data visualization, terminal presentation, and physical indicators.
3. **BAC Project & Tooling Guide** – licensing, versioning, repositories, data formats, SKUs, CLI implementation, configuration and secrets, Marimo structure, Blender extensions, and project-wide engineering conventions. The **BAC Trademark Policy** governs how others may use the BAC name and marks, and the **Rocinante naming note** how the flight software stack is named.

   These guides live in bac-design (https://github.com/buildacubesat/bac-design).
4. **Subsystem READMEs, SDDs, test reports and releases** – current hardware and software implementation.
5. **Issue trackers and project boards** – active work, unresolved engineering questions and near-term priorities.
6. **Storefront and legal pages** – current prices, availability, shipping, terms and company details.

This document should change only when the project's **structure, purpose or major architecture** changes. Routine revisions, release dates and short-term plans should not require an edit here.

1 In One Paragraph

Build a CubeSat (BAC) is an open-source CubeSat platform intended to make real satellite engineering more accessible to students, educators, researchers, makers and small teams. It combines modular spacecraft hardware, structural CAD, the **Rocinante** flight-software stack, development and test tooling, documentation, educational material and commercially available hardware. BAC began as a self-taught engineering project and has grown into a public development program built around a simple premise: useful space hardware should be understandable, modifiable, testable and affordable enough that more people can participate in building it.

BAC is operated commercially through **nihil.space GmbH**, which provides the legal and commercial framework and also undertakes civilian space-engineering work. BAC remains the public project, product and community identity.

2 Why BAC Exists

Commercial CubeSat development hardware is often expensive, closed, vertically integrated or difficult to modify. That is workable for organizations buying a finished solution, but much less useful for people who want to understand the system, replace a subsystem, build a new board, study failure modes or carry knowledge from one project into the next.

BAC takes the opposite approach:

- **Open by default.** Schematics, PCB layouts, mechanical designs, software and documentation are published under established open licenses.

- **Modular rather than monolithic.** Subsystems are intended to be replaceable, extendable and reusable across development configurations.
- **Engineering-grade, not toy-grade.** The project aims toward credible spacecraft engineering practices even when current hardware is still development hardware.
- **Affordable enough to experiment.** Cost reduction matters because iteration is central to learning and engineering.
- **Transparent about maturity.** Experimental, incomplete and unqualified work is labeled as such rather than presented as finished flight hardware.
- **Educational through the process itself.** Design decisions, tests, mistakes and revisions are part of the public record rather than hidden behind the finished product.

The long-term objective is not merely to produce a kit. It is to build an open spacecraft platform and the engineering culture around it.

---

3 Project Principles

3.1 Space for Everyone

BAC treats access to space technology as a participation problem as much as a technical one. The intended audience includes people who may not begin with institutional funding, aerospace credentials or access to proprietary spacecraft platforms.

3.2 Open Source Must Be Usable

Publishing source files is not enough. A useful open platform also needs understandable documentation, reproducible releases, standard interfaces, sourcing information, test procedures and a path for outside contributors to extend it.

3.3 Fail Loudly, Succeed Quietly

A caveat is useful information. WIP, TBD, PRELIMINARY, DO NOT USE and similar labels are part of the project's engineering language, not embarrassments to be polished away. This principle is reflected in the BAC Identity, Interface Design and Project & Tooling guides.

3.4 Build Toward Real Flight Constraints

BAC development hardware is not presented as flight-qualified simply because it uses spacecraft-like form factors or architecture. Nevertheless, the project uses a real spacecraft as the calibration target: interface discipline, configuration control, software quality, testability, failure handling and traceability are developed with eventual flight use in mind.

3.5 Civilian Purpose

nihil.space undertakes civilian work focused on science, education, exploration and related engineering. BAC's published open-source designs are necessarily available to downstream users under their licenses; the civilian-use commitment governs the contracts, partnerships and development direction undertaken by the project and company themselves.

4 Historical Context

Only milestones useful for understanding the project are kept here:

- **2020** – BAC begins as a personal effort to learn the disciplines required to design and build a CubeSat.
- **2024** – the public YouTube development log begins, turning the engineering process into a documented community project.
- **2025** – the first integrated, CubeSat-class prototype, **SN1**, flies on a high-altitude balloon, providing experience with telemetry, operations, environmental testing and post-flight analysis.
- **After SN1** – the project begins a broader architectural maturation: dedicated EPS and carrier hardware, a structured interconnect, and the Rocinante software architecture replace much of the balloon-era prototype stack.

The SN1 architecture remains useful project history, but it should not be cited as the current BAC spacecraft architecture. In particular, its Python/LoRa/CBOR/Firebase telemetry stack was a test-flight solution and has been superseded in the main development path.

5 System Architecture

BAC is best understood as a **platform of replaceable layers**, not one fixed spacecraft configuration.

5.1 Mechanical and Electrical Platform

The hardware program includes:

- CubeSat structures and mechanical parts;
- an electrical power system with battery, charging, regulation, switching, monitoring and protection;
- MicroMod carrier hardware for small- and medium-node subsystem control;
- a Raspberry Pi Compute Module carrier for higher-level computing;
- **bacBus**, the modular board-to-board interconnect using the M.2 Key-M mechanical form factor;
- development utilities such as breadboard/prototyping boards;
- subsystem research and extensions including ADCS, communications, payload interfaces and deployables.

Individual boards move through their own revision and test cycles. **The hardware repository and release notes are authoritative for current revisions and suitability.** A board marked DO NOT USE, experimental or testing should be treated accordingly.

5.2 Rocinante Flight Software

The current software architecture is called **Rocinante** (*Roci* after first mention). Its central architectural split is deliberate:

- **Small nodes** run Zephyr on small, energy-efficient MCUs for deterministic subsystem-level control. These are mainly always-on nodes (like the EPS MCU) and sensor heads.
- **Medium nodes** run Zephyr and NASA F Prime on larger MCUs for more demanding flight tasks like FDIR and COMMS.
- **Large nodes** run Linux and NASA F Prime on higher-capability compute hardware such as the CM5. These are payload nodes.
- Nodes communicate through a shared protocol boundary rather than pretending every processor should run the same framework.

The current main inter-node transport is CSP over CAN, with BAC-defined services for functions such as heartbeat, housekeeping, time, commands, parameters and alerts. CCSDS is used where appropriate for spacecraft/ground framing. Command authentication is designed separately from confidentiality so the system can remain compatible with relevant amateur-radio constraints where those apply.

The architecture is intentionally modular. Specific component versions, node IDs, packet layouts, security details and implementation status belong in the Rocinante SDDs and source repositories rather than this overview.

5.3 Ground and Development Tooling

BAC includes the tooling needed to develop, test and operate the platform: command-line utilities, engineering notebooks, release tooling, visualization, manufacturing helpers and ground-segment work. These tools follow the **BAC Interface Design Guide** for interaction and presentation and the **BAC Project & Tooling Guide** for implementation conventions.

---

6 Maturity and Qualification

BAC distinguishes between **development capability** and **flight qualification**.

Unless a specific release explicitly states otherwise, BAC hardware should be assumed to be intended for development, education, laboratory testing and relevant-environment experimentation rather than orbital flight. The same principle applies to software maturity: an architecture may be designed with flight use in mind without claiming a stable or flight-qualified release.

Qualification is treated as a program of evidence, not a marketing label. It requires increasingly representative hardware, repeatable testing, documented configuration, failure analysis, environmental testing and system-level validation.

A useful shorthand for the project's maturation path is:

bench integration → relevant-environment testing → subsystem maturation → qualification hardware → integrated flight model → orbital demonstration

These are **capability gates, not calendar commitments**. Public dates should only be attached to them when a real program, partner and sufficiently mature plan justify doing so.

---

7 Products and Commercial Model

BAC sells or plans to sell hardware derived from the same open platform used for development. The central product concept is a **CubeSat development kit** that gives users a mechanically and electrically coherent starting point while allowing them to replace compute modules, subsystem boards and payload hardware as their work evolves.

Other product categories include subsystem hardware, education-focused configurations, development utilities and limited merchandise. **Current catalog, price and availability belong on the storefront, not in this document.**

The business model does not depend on keeping the design secret. Commercial value comes from:

- convenient sourcing;
- reliable fabrication and assembly;
- tested configurations;
- integration work;
- documentation;
- support;

- availability;
- and reducing the time between an idea and useful hardware on the bench.

That distinction is central to BAC: the files remain open; the product is the quality and convenience of turning those files into something people can actually use.

8 Open-Source Model

Canonical licensing is defined in the **BAC Project & Tooling Guide**. At project level, the intended split is:

ARTIFACT	LICENSE
Hardware	CERN OHL-S 2.0, mechanical parts included
Software	MIT
Documentation	CC BY-SA 4.0; READMEs and instructions that belong to a design follow its repository
Brand marks	Not openly licensed – BAC Trademark Policy

Older hardware may retain the license under which it was originally released unless it has been explicitly relicensed, as all bac-structure releases were in 2026-10. The license declared by the artifact or repository is authoritative.

Contributors retain their work under the applicable project license rather than assigning ownership simply to participate. The objective is a genuinely extendable ecosystem in which new boards, tools and improvements can exist both in core BAC repositories and in contributor-owned projects that implement BAC interfaces.

9 Community, Documentation and Education

BAC is developed in public. The community layer is not separate from the engineering program; it is one of the ways the platform is tested, explained and improved.

Key public surfaces include:

- the **YouTube development log**, documenting design decisions, failures, tests and progress;
- **Codeberg**, the canonical home for BAC hardware, software, structure and documentation;
- the BAC community chat, used for discussion and contributor coordination;
- **CubeSat Resources**, a related curated collection of learning material;
- project documentation and release artifacts intended to make the hardware reproducible rather than merely viewable.

Contribution is encouraged at more than one level. Focused fixes and improvements can land directly in core repositories; independent modules, tooling and experiments can remain in contributor-owned repositories while still becoming part of the wider BAC ecosystem through documented interfaces.

The project should favor **contributor-ready boundaries** over centralising every idea into one repository.

10 BAC and nihil.space

Build a CubeSat is the project, product and community identity. **nihil.space** provides the legal and commercial framework through which BAC is operated and also undertakes civilian space-engineering work outside BAC.

The separation is intentional:

IDENTITY	PRIMARY ROLE
Build a CubeSat / BAC	Open spacecraft platform, products, documentation and community
Rocinante / Roci	BAC flight-software stack, based on Zephyr and NASA F Prime
nihil.space GmbH	Legal/commercial vehicle and professional engineering practice

BAC-facing communication should remain BAC-led. nihil.space appears where legal, contractual or company context requires it rather than as routine co-branding.

Consulting and product development reinforce each other: engineering work outside BAC can finance and inform the open platform, while BAC provides a public body of real engineering work that demonstrates capability far more effectively than a conventional portfolio.

Current legal structure, addresses, ownership details, contract counterparties and similar corporate information belong in nihil.space's legal/company records rather than this overview.

11 Development Direction

BAC should avoid publishing a long list of calendar promises as its strategic roadmap. The durable direction is better expressed as a set of program goals:

1. **Make the development platform coherent end to end.** Hardware, firmware, flight software, documentation and tooling should work together as one reproducible system.
2. **Increase evidence, not just capability.** Each revision should improve testing, characterisation, traceability and understanding of failure modes.
3. **Mature the platform toward flight use.** Progress from development hardware through relevant-environment testing and qualification rather than skipping directly to a flight claim.
4. **Strengthen open interfaces.** Make it easier for outside contributors to add boards, software and tools without needing to understand the entire system first.
5. **Build a sustainable project.** Hardware sales, education, community support and civilian consulting should fund continued open development without making openness itself the thing being sold back to users.
6. **Demonstrate the platform in space when ready.** An orbital demonstration remains the natural long-term proof of the program, but readiness and opportunity should drive timing rather than an arbitrary date.

Near-term priorities and committed milestones should live in project boards, issues, release plans and public announcements where they can change without making this overview obsolete.

12 What This Overview Deliberately Omits

The previous consolidated summary included useful research but mixed durable project identity with transient operational detail. The following categories are intentionally excluded here:

- exact future launch dates and partner opportunities;
- exact product-release dates and prices;
- board revision tables and release dates;
- commit counts, stars, forks and "last updated" statistics;
- individual pull-request numbers;
- detailed packet sizes, addresses and register-level implementation notes;
- storefront payment-method lists, contact hours and support-inbox plans;
- personal financing figures, batch-margin estimates and tentative grant applications;

- private contributor compensation arrangements;
- a roster of occasional contributors and contacts;
- client-project details not needed to understand BAC;
- speculative long-term business concepts not yet part of BAC's public program;
- issue-by-issue engineering TODO lists and source-conflict registers.

None of these are necessarily unimportant. They are simply better maintained in **live, purpose-specific records** than in the project's main orientation document.

13 Canonical References

SUBJECT	SOURCE
Identity, voice, visual language, Ops/Mission	**BAC Identity Guide**
Interfaces, accessibility, data-viz, UI and indicators	**BAC Interface Design Guide**
Licensing, versioning, repositories, tooling and project conventions	**BAC Project & Tooling Guide**
Hardware revisions and release artifacts	bac-hardware repository
Mechanical design	bac-structure repository
Rocinante architecture and implementation	bac-software repository and SDDs
Documentation	bac-docs repository, docs.buildacubesat.space
Use of the BAC name and marks	**BAC Trademark Policy**
Flight software naming	Rocinante naming note
Current products, prices and availability	buildacubesat.space
Public short links	bac.page
Learning resources	cubesat-resources.space
Professional / legal company context	nihil.space

Canonical source hosting: Codeberg is primary for BAC project source. GitHub is used where a project depends on GitHub-specific infrastructure.

14 Summary

Build a CubeSat is an open, modular spacecraft-development platform whose real product is broader than a set of PCBs. It combines hardware, software, mechanical design, tooling, documentation, education and a public engineering process into one system intended to lower the barrier to meaningful spacecraft development.

The project's credibility depends less on promising a particular launch year than on steadily increasing what can be demonstrated: hardware that works, software that fails safely, interfaces others can build against, tests that can be repeated, documentation that survives the people who wrote it, and a community capable of improving the system independently.

That is the durable roadmap.

15 Version History

VERSION	DATE	CHANGE
0.4	2026-10-29	Moved from Google Drive into bac-design `guides/`. The slogan shortened to "Space for everyone". §0: the Trademark Policy and the Rocinante naming note among the guides, which live in bac-design. §1, §10: nihil.space GmbH as the legal entity. §5.1: the MicroMod carrier serves small and medium nodes. §8: mechanical parts under CERN OHL-S 2.0, documentation follows the design it describes, the marks excluded, the bac-structure relicense. §13: the docs site, the Trademark Policy and the naming note. Headings to Title Case and American spelling throughout.
0.3	2026-08-10	Rewritten as a durable project overview rather than a consolidated status dump. Removed firm future dates, volatile release/commercial details, repository statistics, personal finance, contributor rosters, implementation-level factoids and the large conflict/TODO register. Reframed the roadmap as capability gates and program goals. Updated authority references to the BAC Identity Guide, BAC Interface Design Guide and BAC Project & Tooling Guide.
0.2	2026-07-23	Consolidated corporate and project context and reconciled several source conflicts.
0.1	2026-07-23	Initial compilation from prior project summaries and working references.

Build a CubeSat — Project Overview · v0.4 · 2026-10-29
Source: guides/bac-project-overview.md · exported by bac-design-export 0.1.0
github.com/buildacubesat/bac-design · CC BY-SA 4.0 · the BAC Marks: BAC Trademark Policy